

Research report

POC + hardware

Idea-as-a-Service POC on a Mac M5 and an NVIDIA DGX Spark

How the low-budget POC plan changes when the inference, sandbox and control plane run on hardware you already own: architecture, performance ceilings, budget, an adapted 8-week plan, benchmark protocol and risks.

Prepared September 2026. Companion to **laaS-Low-Budget-POC-Plan** and **mac-m5-dgx-plan/PLAN.md**. No benchmarks were run for this report: speed figures are computed ceilings, and third-party numbers are quoted with their source. Prices and specs should be re-checked before purchase.

1. Executive summary

The POC must answer one question: can an AI pipeline, with three human approvals, turn a one-page idea into a tested web app on a staging URL at an affordable cost? The original plan spends \$220 to \$875 mostly on hosted model APIs and GPU rental. A Mac M5 plus a DGX Spark can replace nearly all of that spend.

Recommendation. Run the POC local-first. The **Spark** serves the large "smart" and "code" models through vLLM or SGLang behind LiteLLM. The **Mac** hosts the control plane, the sandbox, and a small "fast" model natively with MLX. Keep one capped hosted frontier model as an optional escalation path for planning, and decide at the week-6 gate whether it is needed. Expected incremental cost is about \$0 to \$40 over eight weeks, plus your time.

- **Feasible:** a 120B-class MoE model fits in the Spark's 128 GB and is reported at roughly 50 to 70 tokens/s single-stream, enough for a pipeline that makes hundreds of calls per project.
- **Main technical risk:** memory bandwidth, not capacity. Large *dense* models are slow on both machines (section 4). Prefer MoE models and 4-bit weights.
- **Main measurement risk:** a gateway fallback from Spark to the small Mac model would silently lower quality and corrupt the week-6 gate. Log the served model on every call (section 5).
- **Not a scale test:** passing here proves the loop, not the 8-GPU blueprint. Hardware gaps are listed in section 9.

Scope and assumptions

- The POC is the Idea-as-a-Service low-budget plan (idea to PRD to typed contracts to tests to code to verify to repair to staging).
- "DGX" means a **DGX Spark** (GB10 Grace Blackwell, 128 GB unified memory, arm64). A DGX Station or rack DGX changes model sizes, not the layout.
- The Mac M5 variant is unknown, so all tiers are shown. Hardware is treated as already owned: purchase cost is excluded, and section 7 discusses whether to buy.

2. What the POC needs, and how this hardware maps to it

POC substitute (from the plan)	With Mac M5 + DGX Spark	Effect
Hosted open-weight coder API (\$80-200)	Spark serves a 30B-120B MoE coder/general model; Mac serves a 7-14B model	API spend to near \$0; no per-token cost
Local Ollama 7B-14B on existing machine (Tier 0)	Same, but with a much stronger "smart" tier on the Spark	Tier 0 quality rises toward Tier 1
Frontier model for planning and escalation (\$300-400)	Optional capped hosted call, or a large local MoE for plan/review	Decide from week-6 data
Rented GPU benchmarking (\$150)	Measure self-hosting economics directly on the Spark	Removed
Rootless Docker sandbox, no egress	Unchanged. Runs on the Mac or the Spark (both arm64); needs no GPU	No change
State machine + SQLite/Postgres	Unchanged, on the Mac	No change
Schema validation + re-ask	Add JSON-schema guided decoding in vLLM/SGLang where supported, keep the validator as the gate	Fewer retries; validator stays authoritative

POC substitute (from the plan)	With Mac M5 + DGX Spark	Effect
Staging on a small VPS (\$10-20/mo)	Local preview on the LAN or Tailscale; VPS only for a public demo	\$0 to \$20/mo

The 8-GPU inference node, Temporal, Kubernetes, Firecracker and multi-tenant components stay out of scope, exactly as in the POC plan.

3. Hardware comparison

	M5 (base)	M5 Pro	M5 Max	DGX Spark
Role in this POC	Control plane, fast model	Control plane, fast/mid model	Could also host the big model	Inference server
Max unified memory	up to 32 GB*	up to 64 GB	up to 128 GB	128 GB LPDDR5x
Memory bandwidth	153 GB/s*	up to 307 GB/s	up to 614 GB/s	273 GB/s
AI acceleration	Neural Accelerator in each GPU core	same, more cores	same, more cores	5th-gen Tensor Cores, FP4 (about 1 PFLOP sparse FP4, vendor figure)
Software path	MLX, Ollama, LM Studio (native macOS)	same	same	CUDA: vLLM, SGLang, TensorRT-LLM, llama.cpp; NGC containers
Docker GPU access	None (Metal is not exposed)	None	None	Yes, NVIDIA Container Toolkit
Reference price (US)	MacBook Pro from \$1,699	from \$2,199	from \$3,599	about \$4,000-4,700 (varies by report)

Apple figures for M5 Pro and M5 Max are from Apple's March 2026 newsroom release. *M5 base bandwidth and memory ceiling are from Apple's October 2025 launch specification and were not re-fetched. Spark price has changed since launch and is a reported figure. Verify before buying.

Reading the table. The Spark has the same capacity as an M5 Max but under half its bandwidth, so an M5 Max with 128 GB can out-decode a Spark on the same model. The Spark's advantages are CUDA software, FP4 acceleration, batch throughput, containers and being an always-on server. If your Mac is an M5 Max, it is a serious inference node in its own right.

4. Performance analysis

Single-stream decoding reads the active weights once per generated token, so $\text{tokens/s} \leq \text{memory bandwidth} / \text{active bytes per token}$. The chart applies this to four model classes. It ignores compute, KV cache reads and overhead, so real numbers are lower. "No fit" means weights plus 8 GB headroom exceed device memory.

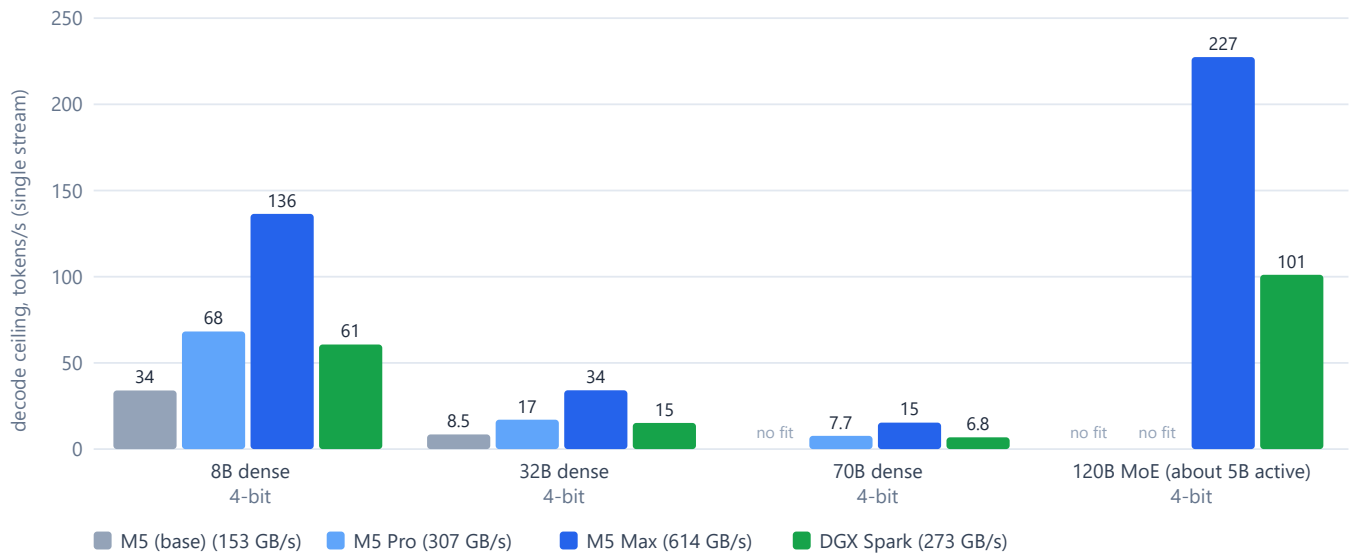


Figure 1. Computed decode ceilings (not measurements). Active bytes: 8B = 4.5 GB, 32B = 18 GB, 70B = 40 GB, 120B MoE = 2.7 GB read per token (about 5B active parameters at 4-bit) with a 66 GB resident footprint.

Model class (tokens/s ceiling)	M5 (base)	M5 Pro	M5 Max	DGX Spark
8B dense, 4-bit	34	68	136	61
32B dense, 4-bit	8.5	17	34	15
70B dense, 4-bit	no fit	7.7	15	6.8
120B MoE (about 5B active), 4-bit	no fit	no fit	227	101

Checking the ceiling against reported results

- For gpt-oss-120b on a Spark the ceiling is about 100 tokens/s. Reported results: about 50 tokens/s (LMSYS, SGLang), about 52 tokens/s (SGLang, MXFP4), and 68.9 tokens/s with a patched vLLM build (community repository). That is roughly 50-70% of the ceiling, a plausible range for well-tuned engines.
- Engine choice matters severalfold: forum and blog reports show the same model much slower on some engines and builds than on others. Treat the serving engine as a benchmarked decision, not a default.
- Aggregate throughput scales with concurrency: the same community report gives about 300 tokens/s aggregate at 30 users. A pipeline that runs several stages or ideas in parallel benefits.
- Dense 70B on the Spark has a ceiling near 7 tokens/s, which is too slow for interactive work. Use a MoE or a 30B-class model for "code" and "smart".

What this means for the POC

A typical run has perhaps 100 to 300 calls, many under 2,000 output tokens. At 30 to 60 tokens/s that is minutes of generation per stage, not hours, so the "under 4 hours from idea to staging" gate is reachable. Prefill on long prompts (PRD plus contracts plus template) is compute-bound and favours the Spark over the base Mac. Measure both (section 8).

5. Architecture and model placement

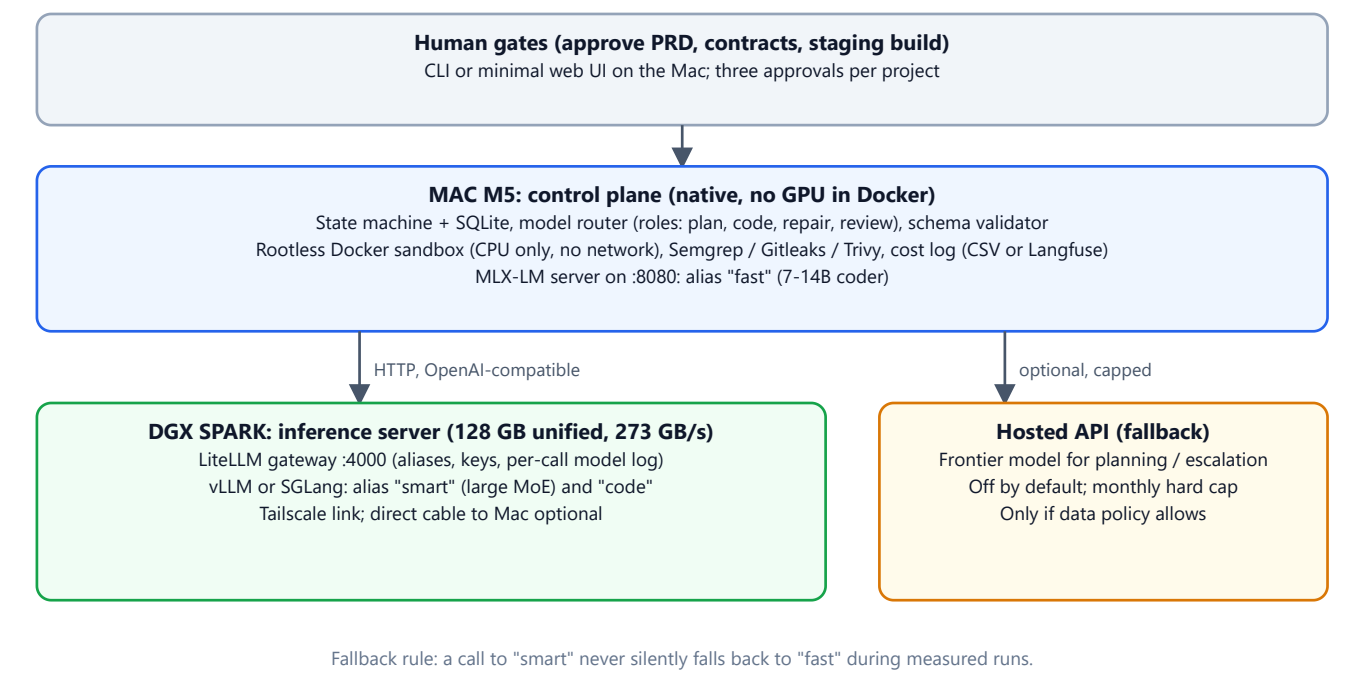


Figure 2. POC topology. The Mac owns control and verification; the Spark owns heavy inference.

Router role	Alias	Where	Model class (examples, verify availability)	Why
plan	smart	Spark	~100B+ MoE (for example gpt-oss-120b class) or capped hosted frontier model	Highest reasoning value per token; few calls
code	code	Spark	30B-class MoE coder (Qwen3-Coder class)	Best quality-to-speed ratio under 273 GB/s
repair	code	Spark	Same as code; escalate to smart on 3rd attempt	Bounded loop, max 3
review	smart	Spark	Different model family from code if possible	Reduces correlated errors
scratch	fast	Mac (MLX)	7-14B coder	Cheap lint, summaries, commit messages; keeps working when the Spark is busy

Setup constraints that decide the design

- **Docker on macOS cannot use the Metal GPU.** Run the Mac model natively (MLX-LM or Ollama); use Docker on the Mac only for the CPU-only sandbox.
- **Both machines are arm64**, so one `linux/arm64` sandbox image runs on either. Prefer NGC containers on the Spark because some CUDA wheels lag on arm64.
- **The Mac cannot join a GPU cluster.** It is a client and edge node. Pooling Mac and Spark for one model (for example exo) is experimental and usually slower than the Spark alone; skip it.
- **Unified memory is shared** with the OS and services on both machines. Budget weights, KV cache, Postgres and sandboxes together; do not set vLLM GPU memory use above about 0.6 to 0.7 while other services run.
- **Network:** Tailscale is enough for token traffic. A direct cable is only useful for copying model weights.

Gate-corruption hazard. The starter `litellm-config.yaml` falls back from `smart` to `fast`. If the Spark is down or slow, runs would succeed or fail on a much weaker model and the week-6 metrics would be meaningless. During measured runs, disable that fallback (fail the stage instead) and record `alias`, actual model, engine, tokens and latency on every call in the cost log.

Structured output

Both vLLM and SGLang can constrain output to a JSON schema during decoding. Use this for the typed contracts to cut re-ask retries, but keep the post-hoc schema validator as the gate: the POC's rule is that schema-invalid output must never reach a later stage.

6. Security and isolation

- Generated code runs only in rootless Docker with no network, CPU, memory and time limits, a read-only root and no host mounts. Never mount secrets or the model cache.
- Do not run the sandbox on the same account that holds API keys. Keep the hosted-API key out of the sandbox environment.
- Expose nothing to the public internet. LiteLLM on :4000 is reachable over Tailscale only; use a master key and per-project virtual keys with budgets.
- Treat LLM output and any fetched web content as untrusted. Prompt-injected specs must not be able to change gate settings or router configuration.
- gVisor is a later hardening step, needed only when running third-party code (out of POC scope).

7. Budget

Item (8 weeks)	Original Tier 1 Lean	Original Tier 2 Standard	Mac + Spark, local-only	Mac + Spark, plus capped frontier
Inference	\$80-120 hosted	\$450-600 hosted + frontier	\$0	\$50-150 capped
GPU benchmarking	-	\$150	\$0 (own Spark)	\$0
Staging / Forgejo	\$10-20/mo	\$10-20/mo	\$0 (LAN) or optional VPS	same
Electricity	-	-	about \$10-25 (Spark about 100-240 W under load; Mac far less)	same
Domain (optional)	about \$12/yr	about \$12/yr	optional	optional
Total incremental	about \$220	about \$875	about \$0-40	about \$60-200

Original tiers are from POC_Budget.csv. Electricity is an estimate at typical US rates; recompute for your tariff. Hardware purchase is excluded.

Should you buy hardware for this POC?

No. If you do not already own the machines, the POC is cheaper on hosted open-weight APIs (\$80-120) than on a \$4,000+ Spark. Buy only if you also need private local inference, fine-tuning, or steady spend above roughly

\$500 a month, which is the plan's own revisit trigger. If you own only the Mac, an M5 Max with 128 GB can serve the "smart" tier by itself; a base M5 should use Tier 1 hosted models for anything above 14B.

8. Adapted 8-week plan and benchmark protocol

Week	Goal (POC plan)	Mac + Spark additions
1	Stack, demo domain, three test ideas, repo, Compose, router with 2 providers	Bring up Tailscale, vLLM/SGLang on Spark, MLX on Mac, LiteLLM. Run the benchmark protocol below. Choose engine and models from data.
2	Idea to PRD to typed contracts with validation	Compare guided decoding vs validate-and-re-ask on retry rate and latency.
3	Tests-first code generation, sandbox, real token cost	Compare "code" candidates on the template task. Record tokens and wall-clock per stage.
4	Repair loop (max 3), Semgrep/Gitleaks, failure taxonomy	Test escalation of the 3rd repair attempt to "smart". Log per-alias fix rate.
5	Human gates, staging deploy, minimal UI or CLI	Deploy preview on Tailscale/LAN; VPS only if a public demo is needed.
6	3 ideas x 3 runs, metrics, go/no-go	Fallback disabled. Report metrics per model configuration.
7-8	If go: second stack, frontier planning, GPU cost benchmark, demo	Compare local-only vs capped-frontier planning quality and cost. Write up the scale gaps (section 9).

Week-1 benchmark protocol

- **Prompts:** three fixed sets (short chat, PRD-sized 3-4k token context, template-plus-contracts 8-12k token context), each run 5 times after a warm-up.
- **Metrics:** time to first token, decode tokens/s, prefill tokens/s, peak memory, and (Spark) power. Record at concurrency 1, 4 and 8.
- **Matrix:** Mac fast model on MLX vs Ollama; Spark candidate models on vLLM vs SGLang vs llama.cpp. Keep quantization and context length identical within a comparison.
- **Quality check:** run the same 10 template tasks on each candidate and count passing generated tests; speed is worthless if pass rate collapses.
- **Output:** a `docs/benchmarks.md` table and the chosen router configuration, committed before week 2.

Week-6 gate (unchanged from the POC plan)

Metric	Pass	Kill or rethink
Ideas reaching a working staging app (of 3)	2 or more	0 or 1
Generated tests passing after repair	85% or more	below 60%
Extra human interventions per project	fewer than 5	10 or more
Cost per project (tokens plus compute)	under \$100	over \$250
Time from idea to staging	under 4 hours	more than 2 days
Schema-invalid output reaching later stages	0	any

With local inference, "cost per project" is dominated by electricity and your time, so also record cost as if the same tokens had been bought from a hosted API. That is the number a customer-facing business case needs.

9. Risks, gaps and open questions

Risk	Mitigation
Memory bandwidth makes dense 70B unusable	MoE and 30B-class models; 4-bit weights; check the ceiling table before choosing a model
Serving-engine performance varies widely on the Spark (new GB10 kernels)	Benchmark vLLM, SGLang and llama.cpp in week 1; pin image tags; keep a fallback engine
Silent fallback to a weaker model corrupts metrics	Disable fallback during gate runs; log actual model on every call
arm64 or CUDA package gaps	NGC containers; check tool support before adopting
Unified-memory contention (model, KV cache, services)	Cap GPU memory fraction, limit context, watch swap and OOM
Local model quality below frontier for planning	Template-first and tests-first design; capped frontier escalation; measure the gap in weeks 7-8
Results do not transfer to the 8-GPU blueprint	Treat as loop validation only. Concurrency, multi-tenant isolation and 70B+ dense serving remain untested
Stale specs and prices	Re-verify hardware prices, model names and engine versions at week 1

Open questions

- Which M5 variant and how much RAM? (Decides whether the Mac can host anything above 14B.)
- Is a hosted frontier fallback allowed by data policy, and what is the monthly cap?
- Is the DGX definitely a Spark, and is it dedicated to this work?
- Which demo domain and stack (recommended: FastAPI or Node with SQLite/Postgres and a simple UI)?

Sources

- [Apple Newsroom: MacBook Pro with M5 Pro and M5 Max \(March 2026\)](#)
- [LMSYS: Optimizing GPT-OSS on NVIDIA DGX Spark](#)
- [Community repository: gpt-oss-120b on one DGX Spark with patched vLLM \(68.9 tok/s single-stream, 300 tok/s at 30 users\)](#)
- [SGLang analysis, SM121 / gpt-oss-120b on DGX Spark](#)
- [Dendro Logic: DGX Spark concurrency benchmark](#)
- Repository files: [idea-as-a-service-plan/poc.html](#), [POC_Budget.csv](#); [mac-m5-dgx-plan/PLAN.md](#) and [starter/](#)

Community benchmark figures come from search summaries of the linked pages and were not independently reproduced.